

AI FEATURE DELIVERY IN REGULATED ENVIRONMENTS

Module 1 — Governed AI Feature Delivery

THE PROBLEM WE'RE SOLVING

AI features behave differently from traditional backend features.

-
-
-
-

THE PROBLEM WE'RE SOLVING

AI features behave differently from traditional backend features.

- Non-deterministic outputs and quality variance
-
-
-

THE PROBLEM WE'RE SOLVING

AI features behave differently from traditional backend features.

- Non-deterministic outputs and quality variance
- Model and prompt version drift over time
-
-

THE PROBLEM WE'RE SOLVING

AI features behave differently from traditional backend features.

- Non-deterministic outputs and quality variance
- Model and prompt version drift over time
- Hidden risk (hallucination, PII, unsafe actions)
-

THE PROBLEM WE'RE SOLVING

AI features behave differently from traditional backend features.

- Non-deterministic outputs and quality variance
- Model and prompt version drift over time
- Hidden risk (hallucination, PII, unsafe actions)
- Audit and governance expectations

THE PROBLEM WE'RE SOLVING

AI features behave differently from traditional backend features.

- Non-deterministic outputs and quality variance
- Model and prompt version drift over time
- Hidden risk (hallucination, PII, unsafe actions)
- Audit and governance expectations

Today: define a safe, reusable architecture.

WHAT "GOVERNED" MEANS

-
-
-
-
-

WHAT "GOVERNED" MEANS

- Traceable requests
-
-
-
-

WHAT "GOVERNED" MEANS

- Traceable requests
- Versioned prompts and models
-
-
-

WHAT "GOVERNED" MEANS

- Traceable requests
- Versioned prompts and models
- Validation gates
-
-

WHAT "GOVERNED" MEANS

- Traceable requests
- Versioned prompts and models
- Validation gates
- Clear responsibility boundaries
-

WHAT "GOVERNED" MEANS

- Traceable requests
- Versioned prompts and models
- Validation gates
- Clear responsibility boundaries
- Repeatable evaluation and release criteria

WHY THIS MATTERS NOW

-
-
-
-

WHY THIS MATTERS NOW

- AI is already in production paths, internal and client-facing.
-
-
-

WHY THIS MATTERS NOW

- AI is already in production paths, internal and client-facing.
- Teams need standards, not one-off implementations.
-
-

WHY THIS MATTERS NOW

- AI is already in production paths, internal and client-facing.
- Teams need standards, not one-off implementations.
- Governance has to be built into delivery, not added later.
-

WHY THIS MATTERS NOW

- AI is already in production paths, internal and client-facing.
- Teams need standards, not one-off implementations.
- Governance has to be built into delivery, not added later.
- Reusable patterns reduce risk and speed up future features.

TRADITIONAL VS AI FEATURE: OUTPUT BEHAVIOR

Traditional feature	AI feature	-----	-----
-----	Deterministic	Probabilistic / variable	

TRADITIONAL VS AI FEATURE: OUTPUT BEHAVIOR

Traditional feature	AI feature	-----	-----
-----	Deterministic	Probabilistic / variable	

Traditional: Same input always gives the same output.

TRADITIONAL VS AI FEATURE: OUTPUT BEHAVIOR

Traditional feature	AI feature	-----	-----
-----		Deterministic	Probabilistic / variable

Traditional: Same input always gives the same output.

AI: Output can vary with each request, even for identical input.

TRADITIONAL VS AI FEATURE: OUTPUT BEHAVIOR

Traditional feature	AI feature	-----	-----
-----		Deterministic	Probabilistic / variable

Traditional: Same input always gives the same output.

AI: Output can vary with each request, even for identical input.

This uncertainty impacts testability, reliability, and user trust.

TRADITIONAL VS AI FEATURE: TESTING APPROACH

	Traditional feature	AI feature
--	---------------------	------------

	-----	-----
-	Unit/integration mostly enough	Needs eval datasets + quality thresholds

TRADITIONAL VS AI FEATURE: TESTING APPROACH

	Traditional feature	AI feature
	-----	-----
-	Unit/integration mostly enough	Needs eval datasets + quality thresholds

Traditional: Unit and integration tests are typically sufficient.

TRADITIONAL VS AI FEATURE: TESTING APPROACH

| Traditional feature | AI feature |

| ----- | -----
- || Unit/integration mostly enough | Needs eval datasets +
quality thresholds |

Traditional: Unit and integration tests are typically sufficient.

AI: Requires realistic evaluation datasets and measurable
quality targets.

TRADITIONAL VS AI FEATURE: TESTING APPROACH

| Traditional feature | AI feature |

| ----- | -----
- || Unit/integration mostly enough | Needs eval datasets +
quality thresholds |

Traditional: Unit and integration tests are typically sufficient.

AI: Requires realistic evaluation datasets and measurable
quality targets.

Success is about overall averages, not just pass/fail per case.

TRADITIONAL VS AI FEATURE: CHANGE RISK

Traditional feature	AI feature	-----	-----
-----	Mostly code changes	Code + prompt	
	+ model + data drift		

TRADITIONAL VS AI FEATURE: CHANGE RISK

Traditional feature	AI feature	-----	-----
-----	Mostly code changes	Code + prompt	
	+ model + data drift		

Traditional: Most risk comes from code changes and deployments.

TRADITIONAL VS AI FEATURE: CHANGE RISK

Traditional feature	AI feature	-----	-----
-----	Mostly code changes	Code + prompt	
	+ model + data drift		

Traditional: Most risk comes from code changes and deployments.

AI: Risk sources include prompt edits, new model versions, and data shifts.

TRADITIONAL VS AI FEATURE: CHANGE RISK

Traditional feature	AI feature	-----	-----
-----		Mostly code changes	Code + prompt + model + data drift

Traditional: Most risk comes from code changes and deployments.

AI: Risk sources include prompt edits, new model versions, and data shifts.

Change and drift can happen outside of mainline code, requiring new controls.

TRADITIONAL VS AI FEATURE: OBSERVABILITY NEED

| Traditional feature | AI feature | | ----- | -----
----- | | Logs and metrics | Logs, traces,
prompt/model provenance |

TRADITIONAL VS AI FEATURE: OBSERVABILITY NEED

Traditional feature	AI feature	-----	-----
-----		Logs and metrics	Logs, traces, prompt/model provenance

Traditional: Basic logs and service metrics usually suffice for diagnosis.

TRADITIONAL VS AI FEATURE: OBSERVABILITY NEED

Traditional feature	AI feature	-----	-----
-----		Logs and metrics	Logs, traces, prompt/model provenance

Traditional: Basic logs and service metrics usually suffice for diagnosis.

AI: Also must record which prompt and which model generated each output.

TRADITIONAL VS AI FEATURE: OBSERVABILITY NEED

Traditional feature	AI feature	-----	-----
-----		Logs and metrics	Logs, traces, prompt/model provenance

Traditional: Basic logs and service metrics usually suffice for diagnosis.

AI: Also must record which prompt and which model generated each output.

This provenance is key for debugging, trust, and regulatory reviews.

Key point: AI features require a broader and deeper engineering contract.

*Key point: AI features require a broader
and deeper engineering contract.*

Same engineering discipline — expanded control surface.

THE AI REQUEST LIFECYCLE

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

THE AI REQUEST LIFECYCLE

1. User or system submits input.
- 2.
- 3.
- 4.
- 5.
- 6.

THE AI REQUEST LIFECYCLE

1. User or system submits input.
2. Backend workflow prepares context and policy constraints.
- 3.
- 4.
- 5.
- 6.

THE AI REQUEST LIFECYCLE

1. User or system submits input.
2. Backend workflow prepares context and policy constraints.
3. Gateway routes model call and records trace metadata.
- 4.
- 5.
- 6.

THE AI REQUEST LIFECYCLE

1. User or system submits input.
2. Backend workflow prepares context and policy constraints.
3. Gateway routes model call and records trace metadata.
4. Model returns structured/unstructured response.
- 5.
- 6.

THE AI REQUEST LIFECYCLE

1. User or system submits input.
2. Backend workflow prepares context and policy constraints.
3. Gateway routes model call and records trace metadata.
4. Model returns structured/unstructured response.
5. Post-call validation and safety checks run.
- 6.

THE AI REQUEST LIFECYCLE

1. User or system submits input.
2. Backend workflow prepares context and policy constraints.
3. Gateway routes model call and records trace metadata.
4. Model returns structured/unstructured response.
5. Post-call validation and safety checks run.
6. Frontend presents result with confidence/review affordances.

ARCHITECTURE BOUNDARIES (COURSE STACK)

-
-
-
-
-

ARCHITECTURE BOUNDARIES (COURSE STACK)

- TanStack frontend: interaction, review, uncertainty display
-
-
-
-

ARCHITECTURE BOUNDARIES (COURSE STACK)

- **TanStack frontend:** interaction, review, uncertainty display
- **NestJS backend:** orchestration, policy, deterministic workflow steps
-
-
-

ARCHITECTURE BOUNDARIES (COURSE STACK)

- **TanStack frontend:** interaction, review, uncertainty display
- **NestJS backend:** orchestration, policy, deterministic workflow steps
- **LLM gateway:** routing, traceability, auditability controls
-
-

ARCHITECTURE BOUNDARIES (COURSE STACK)

- **TanStack frontend:** interaction, review, uncertainty display
- **NestJS backend:** orchestration, policy, deterministic workflow steps
- **LLM gateway:** routing, traceability, auditability controls
- **Model provider:** inference capability, latency/cost trade-offs
-

ARCHITECTURE BOUNDARIES (COURSE STACK)

- **TanStack frontend:** interaction, review, uncertainty display
- **NestJS backend:** orchestration, policy, deterministic workflow steps
- **LLM gateway:** routing, traceability, auditability controls
- **Model provider:** inference capability, latency/cost trade-offs
- **Evaluation layer:** quality checks and regression detection

WHERE LOGIC SHOULD LIVE

| Put it in... | Typical responsibilities | Example |

-----	-----
-----	Backend code Workflow sequence, retries, policy branching Retry if confidence score below threshold Prompt assets in code Task instructions, output format constraints Extraction schema and field definitions Config Model selection, thresholds, feature flags Model name, confidence threshold, region Infrastructure Deployment policy, secrets, regional routing, monitoring EU/US routing rules, secrets management

Rule: avoid hiding business rules inside prompts only.

RISK CATEGORIES TO DESIGN FOR

-
-
-
-

RISK CATEGORIES TO DESIGN FOR

- **Hallucination:** plausible but incorrect output
-
-
-

RISK CATEGORIES TO DESIGN FOR

- **Hallucination:** plausible but incorrect output
- **Data exposure:** leakage of PII or sensitive context
-
-

RISK CATEGORIES TO DESIGN FOR

- **Hallucination:** plausible but incorrect output
- **Data exposure:** leakage of PII or sensitive context
- **Trace gaps:** no explainable path from input to decision
-

RISK CATEGORIES TO DESIGN FOR

- **Hallucination:** plausible but incorrect output
- **Data exposure:** leakage of PII or sensitive context
- **Trace gaps:** no explainable path from input to decision
- **Operational drift:** prompt/model changes with no controls

REGULATED ENVIRONMENT EXPECTATIONS

-
-
-
-

REGULATED ENVIRONMENT EXPECTATIONS

- You need evidence for how decisions were produced.
-
-
-

REGULATED ENVIRONMENT EXPECTATIONS

- You need evidence for how decisions were produced.
- You need repeatable release criteria, not judgment calls.
-
-

REGULATED ENVIRONMENT EXPECTATIONS

- You need evidence for how decisions were produced.
- You need repeatable release criteria, not judgment calls.
- You need clear controls for fallback and rollback.
-

REGULATED ENVIRONMENT EXPECTATIONS

- You need evidence for how decisions were produced.
- You need repeatable release criteria, not judgment calls.
- You need clear controls for fallback and rollback.
- You need region-aware operations where required (for example EU/US).

GOVERNANCE IS NOT "SLOW"

Common fear: "Governance will slow us down."

-
-
-

GOVERNANCE IS NOT "SLOW"

Common fear: "Governance will slow us down."

- Without standards: every feature reinvents controls.
-
-

GOVERNANCE IS NOT "SLOW"

Common fear: "Governance will slow us down."

- Without standards: every feature reinvents controls.
- With standards: teams ship faster using reusable modules.
-

GOVERNANCE IS NOT "SLOW"

Common fear: "Governance will slow us down."

- Without standards: every feature reinvents controls.
- With standards: teams ship faster using reusable modules.
- Auditability and safety become defaults, not special projects.

GOVERNANCE IS NOT "SLOW"

Common fear: "Governance will slow us down."

- Without standards: every feature reinvents controls.
- With standards: teams ship faster using reusable modules.
- Auditability and safety become defaults, not special projects.

Governance done well is a delivery accelerator.

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

-
-
-
-
-

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

- Accepts document/text input
-
-
-
-

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

- Accepts document/text input
- Extracts structured information
-
-
-

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

- Accepts document/text input
- Extracts structured information
- Classifies/routes results
-
-

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

- Accepts document/text input
- Extracts structured information
- Classifies/routes results
- Applies validation and guardrails
-

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

- Accepts document/text input
- Extracts structured information
- Classifies/routes results
- Applies validation and guardrails
- Surfaces outputs in a reviewable UI

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

- Accepts document/text input
- Extracts structured information
- Classifies/routes results
- Applies validation and guardrails
- Surfaces outputs in a reviewable UI

This scenario was chosen deliberately: it combines extraction, classification, validation, and review — touching every layer of the governed architecture.

CORE SCENARIO FOR THIS COURSE

Build an AI-powered document-processing feature that:

- Accepts document/text input
- Extracts structured information
- Classifies/routes results
- Applies validation and guardrails
- Surfaces outputs in a reviewable UI

This scenario was chosen deliberately: it combines extraction, classification, validation, and review — touching every layer of the governed architecture.

The same feature thread runs through all eight modules.

WHAT WE DELIVER BY THE END OF MODULE 1

-
-
-
-

WHAT WE DELIVER BY THE END OF MODULE 1

- Shared architecture diagram
-
-
-

WHAT WE DELIVER BY THE END OF MODULE 1

- Shared architecture diagram
- Agreed request lifecycle and control points
-
-

WHAT WE DELIVER BY THE END OF MODULE 1

- Shared architecture diagram
- Agreed request lifecycle and control points
- Initial definition of "governed" for this organisation
-

WHAT WE DELIVER BY THE END OF MODULE 1

- Shared architecture diagram
- Agreed request lifecycle and control points
- Initial definition of "governed" for this organisation
- A baseline pattern to reuse in modules 2–8

SUMMARY

- 1.
- 2.
- 3.
- 4.

SUMMARY

1. **AI features are different:** they need broader engineering controls.
- 2.
- 3.
- 4.

SUMMARY

1. **AI features are different:** they need broader engineering controls.
2. **Governed delivery is practical:** boundaries, traceability, validation.
- 3.
- 4.

SUMMARY

1. **AI features are different:** they need broader engineering controls.
2. **Governed delivery is practical:** boundaries, traceability, validation.
3. **Architecture matters early:** where logic lives determines risk.
- 4.

SUMMARY

1. **AI features are different:** they need broader engineering controls.
2. **Governed delivery is practical:** boundaries, traceability, validation.
3. **Architecture matters early:** where logic lives determines risk.
4. **This is the foundation** for workflows, guardrails, UX, evals, and deployment.

BRIDGE TO MODULE 2

What we've learned:

-
-

What's next:

-
-

BRIDGE TO MODULE 2

What we've learned:

- Where responsibilities should sit across the stack.
-

What's next:

-
-

BRIDGE TO MODULE 2

What we've learned:

- Where responsibilities should sit across the stack.
- Which risks and controls must be explicit from day one.

What's next:

-
-

BRIDGE TO MODULE 2

What we've learned:

- Where responsibilities should sit across the stack.
- Which risks and controls must be explicit from day one.

What's next:

- Implement the backend workflow pattern in NestJS.
-

BRIDGE TO MODULE 2

What we've learned:

- Where responsibilities should sit across the stack.
- Which risks and controls must be explicit from day one.

What's next:

- Implement the backend workflow pattern in NestJS.
- Your first task: build the NestJS service skeleton that owns the orchestration boundary. You already know what it's responsible for — now you'll build it.

BRIDGE TO MODULE 2

What we've learned:

- Where responsibilities should sit across the stack.
- Which risks and controls must be explicit from day one.

What's next:

- Implement the backend workflow pattern in NestJS.
- Your first task: build the NestJS service skeleton that owns the orchestration boundary. You already know what it's responsible for — now you'll build it.

Module 2 turns this architecture into reusable implementation structure.

QUESTIONS?

Module 1 — Governed AI Feature Delivery